

Programming for Data Science - COVID-19 Dataset

The objective of this project is to clean and prepare a COVID dataset, analyze and visualize key relationships in the data using object-oriented programming(OOP) design.

Team Members

- Lusine Torikyan
- Rozeta Gevorgyan
- Nelli Sargsyan

Project Content

- ReadMe
- Sources
 - data_analysis.py
 - vizualizations.py
- Graphs *#This folder is not present before running the code. After we run the code, the folder will be created in the package, and the graphs will be added in it.*
 - Barplot
 - Boxplot
 - Bubble Plot
 - Density Plot
 - Line Chart
 - Scatterplot
- main.py
- covid_dataset_ready.csv
- cleaned_data.csv

Used Python Libraries

- Pandas
- Numpy
- Matplotlib
- Seaborn

Data Cleaning and Preprocessing

COVID Data Types:

```
iso_code      object
year          int64
date          object
total_cases   float64
total_deaths  float64
vaccinated_rate float64
fully_vaccinated_rate float64
booster_rate  float64
population    float64
country       object
cases_per_million float64
deaths_per_million float64
dtype: object
```

Initial state and missing values (shape = 744, 12) Final State - after cleaning

iso_code	0	iso_code	0
year	0	year	0
date	39	total_cases	0
total_cases	48	total_deaths	0
total_deaths	48	vaccinated_rate	0
vaccinated_rate	284	fully_vaccinated_rate	0
fully_vaccinated_rate	312	population	0
booster_rate	452	country	0
population	102	cases_per_million	0
country	0	deaths_per_million	0
cases_per_million	108		
deaths_per_million	108		
dtype: int64			

The objective for this stage of the project was to handle the missing values in the dataset and prepare it for the later stages to have a clearer analysis and visualization.

Data Cleaning and Preprocessing Steps:

1. In our initial state of dataset, we had many rows with more than 8 missing values. We dropped those rows, as there were too many unknown data points
2. As the **date** contained the **year** column, we decided to remove it (the month and day were the same for all columns - Dec 31)
3. **Booster_rate** column had 452/744 missing values, we dropped that column
4. After dropping the rows with mostly unknown values, we were able to handle the missing values with filling techniques.
 1. We handled missing values in **total_cases** and **total_deaths** columns by filling them out with their average values, and **population**, **cases_per_million** and **deaths_per_million** by their median values.
 2. In **Vaccinated_rate** and **fully_vaccinated_rate** columns, most missing values were present for the year 2020. They had either missing values, 0-s, or numbers between 0 - 0.35. So we handled the missing values in **vaccinated_rate** by filling them out using `np.random.uniform(0, 0.35)`, and in **fully_vaccinated_rate** by multiplying the previous results by 0.9, as that was the average ratios for the column values for 2021 and 2022.

Data Analysis

By implementing OOP-s main principles, the class DataAnalysis was created in order to have all of the methods (functions) within one class. The dataset became an attribute of the class, which was stored as (self.df), which will assist in allowing all methods to have a direct access to it.

The class applies three main analysis techniques : Grouping and aggregation, Statistical summary and Correlation analysis.

1 - Correlation Analysis

1.1 vaccination_death_correlation_rate - This analyzes whether countries with high vaccination rates had more/fewer death rates, by first computing the average rate of vaccination and by comparing it with each country's vaccination rate against it.

1.2 population_cases_correlation - This examines whether countries with bigger populations had more/less total cases. The way it is doing the computations is similar to the **1.1**.

1.3 vaccination_total_cases_correlation_rate - This analysis provides whether countries with higher vaccination rate had more/fewer total cases. The computation is done by applying the same logic for 1.1.

2 - Statistical Summary

2.1 statistical_summary - The following method provides for the four attributes: total_cases, total_death_rate, deaths_per_million, fully_vaccinated_rate.

2.2 Additional summary - median was selected as additional measure as it is not very highly influenced by extreme values.

3 - Aggregation/Grouping

3.1 aggregation_by_year - Groups the data set by the unique years and calculates the average for total cases and deaths for each specific year, which assists us with understanding how covid-19 evolved over those specific years.

Data Visualization

Our project also includes a class for making meaningful visualizations. In this class, we made 6 methods which create visualizations on correlations between data features and summaries of data. Three of the visualizations were made with Matplotlib and the other three were made with Seaborn. Our visualizations and the methods we created them with are as follows:

1. **connect_deaths_vaccine()** - Uses Matplotlib to create a visualization, showing the correlation between number of deaths and number of cases of Covid - 19.

2. **show_deaths_p_million_per_country()** - Filters the top 10 countries with highest the death number and uses seaborn to create boxplots for each one of them summarizing statistics about total deaths.

3. **show_deaths_p_million_per_country_bars()** - Uses Matplotlib to create a barplot summarizing total cases for the top 10 countries with the highest number of total cases.

4. **correlate_total_case_and_death()** - Uses Matplotlib to create a bubble plot which correlates total cases and total deaths. The size of the bubble depends on the population of a specific country.

5. **density_plot()** - Uses Seaborn to create a kdeplot which shows the distributions of total cases in 2020, 2021, 2022, in countries, which have had the top 10 highest death numbers.

6. **line_chart()** - Implements time-series analysis and shows how the number of cases grew from 2020 to 2022. This chart is for the top 10 countries with the highest death rates.

Overall, the visualizations told us a lot about the Covid-19 statistical patterns.